

HTTPS authentication security report

A working Node.js frontend and backend now communicate exclusively over HTTPS. The application requires an email address at signup, stores it using authenticated AES encryption, and stores only a salted scrypt password hash.

Verified results

All 20 automated security checks passed. A separate Chrome browser run completed signup, logout, login, and authenticated profile retrieval with certificate checking enabled. TLS 1.3 negotiated TLS_AES_256_GCM_SHA384 on both services.

Scope and evidence status

This report documents a local development implementation on Windows. The certificate is signed by a generated local CA trusted for this user with permission; it is not a publicly issued production certificate.

Wireshark and Npcap were not installed during assessment. Windows pktmon reported Access is denied. Consequently, genuine Wireshark capture files, packet analysis and packet screenshots are PENDING. Browser and automated TLS evidence below do not substitute for that requirement.

Artifacts and reproduction

```
npm start
# Frontend: https://localhost:3443
# Backend:  https://localhost:4443
# Stop servers before testing:
npm test
# After installing Wireshark + Npcap:
./scripts/capture-poc.ps1
```

Evidence: output/evidence/poc-results.json, browser-results.json, signup.png, login.png, implementation-excerpt.png. This PDF is an implementation report with the packet-capture section explicitly incomplete.

TLS configuration and verified PoC

HTTPS-only listeners and certificate

Both services use `https.createServer`, bind to `127.0.0.1`, require TLS 1.2 or TLS 1.3 and allow explicit AEAD cipher suites. Neither runs an HTTP listener. Browser fetch calls target `https://localhost:4443`; the frontend CSP allows that HTTPS API endpoint.

```
https.createServer({
  key: tlsKey, cert: tlsCert,
  minVersion: 'TLSv1.2', maxVersion: 'TLSv1.3',
  ciphers: /* TLS 1.3 and ECDHE-RSA AES-GCM suites */
}, handler)
```

Certificate: RSA 3072-bit leaf, CN=localhost. SANs: localhost, 127.0.0.1 and ::1. Issuer: Secure Auth Local Development CA. Valid from 7 October 2026 to 5 January 2027 (UTC). OpenSSL chain and hostname verification returned OK.

Observed test results

```
Frontend certificate chain and hostname verified
status: PASS; authorized: True
protocol: TLSv1.3
cipher: TLS_AES_256_GCM_SHA384
```

```
Signup over verified TLS
status: PASS; authorized: True
protocol: TLSv1.3
cipher: TLS_AES_256_GCM_SHA384
```

```
Login over verified TLS
status: PASS; authorized: True
protocol: TLSv1.3
cipher: TLS_AES_256_GCM_SHA384
```

The Node client supplies `local-ca.pem` as its CA and sets `rejectUnauthorized=true`. No certificate or hostname checks are disabled. Plain HTTP probes on ports 3443 and 4443 received no HTTP response.

TLS encrypts HTTP bodies and authenticates the server. Capturers without session secrets should see encrypted records; IP addresses, ports, sizes, timing and some handshake metadata remain observable.

Signup over trusted HTTPS

The screenshot displays a web application titled "Secure Authentication Demo" with a dark blue header. A badge in the top left corner indicates "HTTPS / TLS 1.2+". Below the header, a message states: "Frontend and API are HTTPS-only. Email is encrypted with AES-256-GCM and passwords are protected with crypt." The main content area features two white boxes: "Create account" and "Login". The "Create account" box contains an "Email" field with the value "browser.1791340798245@example.com", a "Password" field with a placeholder "Minimum 12 characters", and a "Create secure account" button. Below this, a green message says "Signup successful.". The "Login" box contains an "Email" field with the value "student@example.com", a "Password" field, and a "Login securely" button. At the bottom, an "Authenticated session" box shows the user is authenticated as "browser.1791340798245@example.com" with "Check /api/me" and "Logout" buttons. A footer note reads: "Open browser developer tools and confirm every request uses https://."

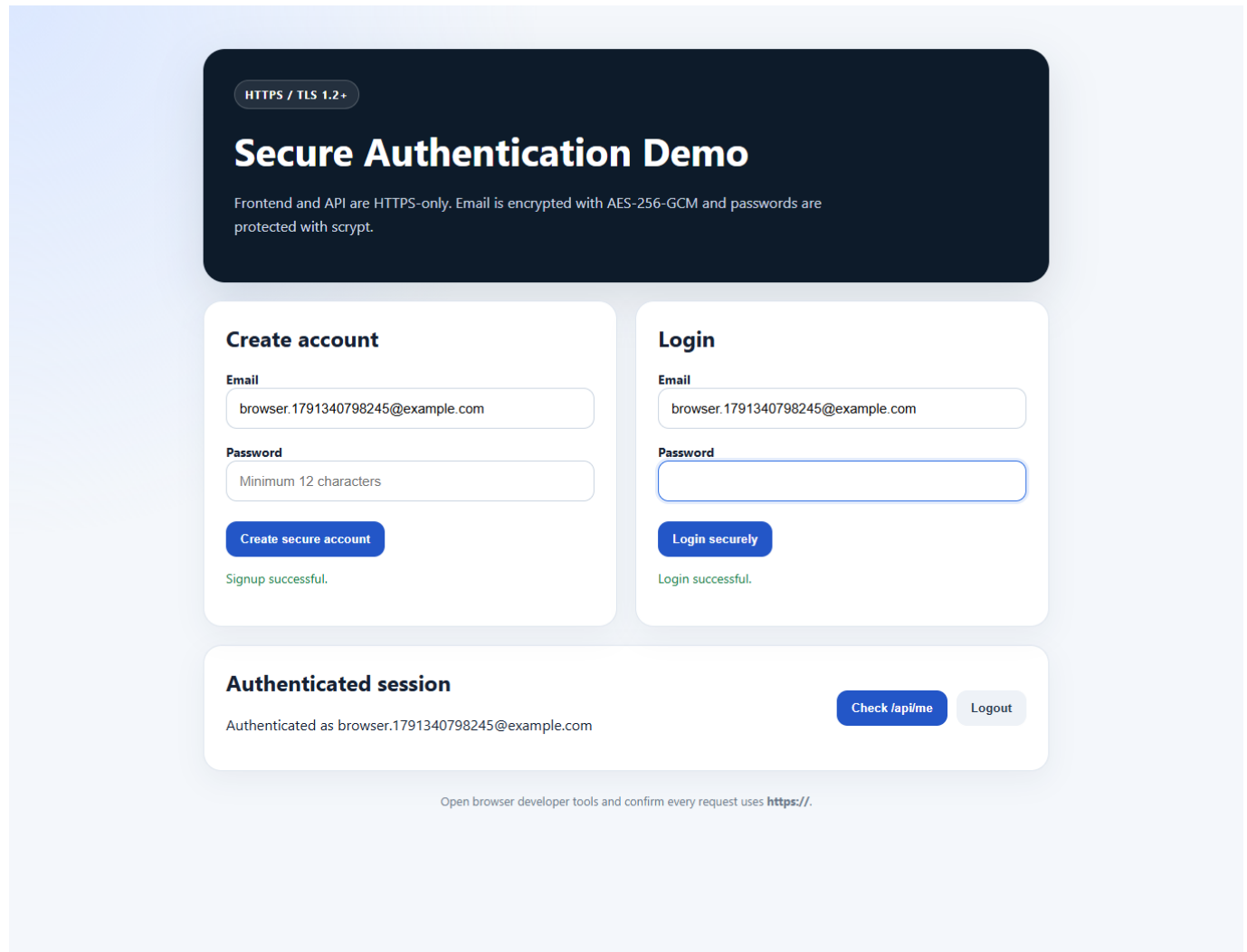
Actual Chrome screenshot after a synthetic account signup. The page reports Signup successful and displays the authenticated email. Password inputs were cleared before capturing the screenshot. Certificate checks were enabled; no HTTPS interstitial was bypassed.

Validation and session controls

Both HTML and backend require email input. Backend validation checks ASCII email syntax, local/domain structure and lengths; it rejects invalid addresses and passwords outside 12-128 characters. Syntax validation does not confirm mailbox ownership.

A 256-bit random session token is sent in a `__Host-sid` cookie: `Secure; HttpOnly; SameSite=Strict; Path=/; Max-Age=1800`. POST requests require the exact allowed origin and JSON content type. API responses are not cached.

Login and authenticated profile



Actual Chrome screenshot after logout, then login using the same synthetic account. Login successful and the authenticated email show that the complete browser flow worked. The API derives a lookup index, verifies the salted hash and issues a fresh session token.

Recorded browser network evidence

https://localhost:4443/api/signup - HTTP status 201; browser security protocol TLS 1.3; issuer Secure Auth Local Development CA.

https://localhost:4443/api/login - HTTP status 200; browser security protocol TLS 1.3; issuer Secure Auth Local Development CA.

Automated checks also rejected a wrong password, invalidated the old session on logout, confirmed profile access after login and verified token rotation. Browser network metadata is endpoint evidence, not a Wireshark packet capture.

Encrypted storage and implementation

```
IMPLEMENTATION EXCERPT | backend/security.js
AES email encryption / scrypt password hashing

export function encryptEmail(email, key) {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const ciphertext = Buffer.concat([cipher.update(email, 'utf8'), cipher.final()]);
  const tag = cipher.getAuthTag();
  return {
    iv: iv.toString('base64'),
    ciphertext: ciphertext.toString('base64'),
    tag: tag.toString('base64')
  };
}

export async function hashPassword(password) {
  const salt = crypto.randomBytes(16);
  const derived = await new Promise((resolve, reject) => {
    crypto.scrypt(password, salt, 64, { N: 131072, r: 8, p: 1, maxmem: 256 * 1024 * 1024 },
    (err, key) => {
      if (err) reject(err); else resolve(key);
    });
  });
  return
  `scrypt$131072$8$1${salt.toString('base64')}${Buffer.from(derived).toString('base64')}`;
}
```

Faithful rendered source excerpt from backend/security.js. AES-256-GCM uses a fresh 12-byte IV for every encryption and stores ciphertext, IV and a 16-byte tag. The tag makes modification detectable. Independent HMAC-SHA-256 indexing supports lookup without storing plaintext email.

Password and stored-record evidence

```
emailEncrypted.ciphertext: zeam5289FBh/qP3fdydnioVZBJ/9VGim0l3TrHU=...
emailEncrypted.iv: UQS6eRd36XLWCwe1
emailEncrypted.tag: rsPlbAXadx2+yzNkSjrzqg==
passwordHash prefix: scrypt$131072$8$1$...
plaintext email stored: NO
plaintext password stored: NO
```

scrypt uses N=131072, r=8, p=1 (about 128 MiB memory), a random 16-byte salt and 64-byte derived hash. Login compares hashes with timingSafeEqual; hashing is one-way and salts prevent identical-password hashes. A stored hash still permits offline guessing. AES email encryption is reversible only with the encryption key.

Tests confirmed random ciphertext on repeated encryption, rejected a changed GCM tag, and found neither the test email nor password in the stored record. Encryption keys and TLS private keys are omitted from this report.

Wireshark packet evidence: pending

No genuine Wireshark packet capture or packet screenshots are available from this machine yet. This section provides the exact procedure for collecting the remaining evidence; it makes no claim that packet searches have been performed.

Capture the real signup and login traffic

1. Install official Wireshark with Npcap loopback support. Start npm start. Clear SSLKEYLOGFILE and the Wireshark TLS secret-log setting; do not export session secrets for this capture.
2. Run scripts/capture-poc.ps1. It selects NPF_Loopback, filters TCP ports 3443/4443 and captures for 45 seconds. During that interval, load the frontend, sign up with unique synthetic credentials, logout and login. Save auth-flow.pcapng.

```
Capture filter: tcp port 3443 or tcp port 4443
Display filter: tcp.port == 3443 || tcp.port == 4443
Handshake: tls.handshake.type == 1 ||
           tls.handshake.type == 2
Data: tls.record.content_type == 23
```

Required screenshots and packet analysis

3. Screenshot ServerHello and expand supported_versions (0x0304 = TLS 1.3) and the selected cipher suite. A legacy record version of 0x0303 alone does not prove TLS 1.2. Decode the two ports as TLS if needed.
4. Screenshot encrypted Application Data and packet bytes for the signup/login time interval. Record packet numbers and TCP streams. TLS 1.3 also carries encrypted handshake messages inside records with outer type 23; correlate timing and flow before labeling data.
5. Find Packet > Packet bytes > String: search the exact password and email. Show the no-match result and search settings. Follow TCP Stream to show unreadable ciphertext. Stored AES ciphertext is not sent to the browser; a no-match for that value alone is not useful proof.
6. Retain the original pcapng, SHA-256 hash, interface, time interval, test sequence, Wireshark version and screenshots. A no-match search alone is insufficient; combine verified handshake, complete capture and successful application flow.

Verification, limits and deployment

Security checks completed

Frontend certificate chain and hostname verified; Invalid email rejected; Short password rejected; Untrusted origin rejected; Malformed JSON rejected; Signup over verified TLS; Secure HttpOnly SameSite host cookie; Authenticated session returns decrypted email; Duplicate signup rejected; Wrong password rejected; Logout invalidates session; Login over verified TLS; Login rotates token; Login session works; No plaintext email or password stored; Stored script hash verifies; Random IV changes ciphertext; AES-GCM tampering rejected; Plain HTTP rejected on 3443; Plain HTTP rejected on 4443.

Deployment and key management

For production, use a public CA certificate for the actual domain, a full certificate chain and automated renewal. Protect private keys and AES/index secrets outside source control. Replace the frontend API/CSP URLs and allowed origin together. Never disable certificate validation.

Use Windows ACLs to restrict .env, key and data access; POSIX mode 0600 does not establish Windows ACLs. Preserve secure key backups and plan re-encryption for rotation. Use a transactional database and shared session/rate-limit storage for multiple instances. The JSON store and in-memory sessions are a single-process demo.

Remaining limits: email ownership is not verified; duplicate signup responses can disclose account existence; there is no password recovery or MFA; process restart clears sessions. TLS protects traffic against observers without keys; compromised endpoints or exposed session secrets can reveal plaintext.

Evidence provenance

Node v22.17.0; test run 2026-10-07T02:37:45.010Z. Browser screenshots were generated from actual local Chrome interactions. Source excerpt was rendered directly from application code. Wireshark/Npcap were absent and no packet evidence was generated.

Primary references

Node.js Crypto: <https://nodejs.org/api/crypto.html>

Node.js TLS: <https://nodejs.org/api/tls.html>

Wireshark download: <https://www.wireshark.org/download.html>

Wireshark TLS analysis: <https://wiki.wireshark.org/TLS>

Wireshark Npcap: <https://wiki.wireshark.org/NPcap/>